

Title

Design of a Watchdog Timer-Based Hardware Timeout Mechanism and Evaluation of Its Role in System Recovery

Objective

To design and implement a Watchdog Timer (WDT) based hardware timeout mechanism using Embedded C. The program demonstrates how a watchdog timer detects software failures and automatically initiates system recovery by simulating a system reset.

Problem Statement

Embedded systems may become unresponsive due to software bugs, infinite loops, or hardware faults. A Watchdog Timer is used to monitor the system. If the application fails to refresh the watchdog within a specified time, the watchdog generates a timeout and resets the system automatically, improving system reliability.

Algorithm

1. Start the program.
2. Initialize the watchdog timer.
3. Execute the main application.
4. Refresh the watchdog after each successful execution cycle.
5. Simulate a software failure by stopping the watchdog refresh.
6. Monitor the watchdog counter.

7. If the timeout limit is reached, simulate a system reset.
8. Continue execution.
9. Stop the program.

Algorithm Using Real Life Example

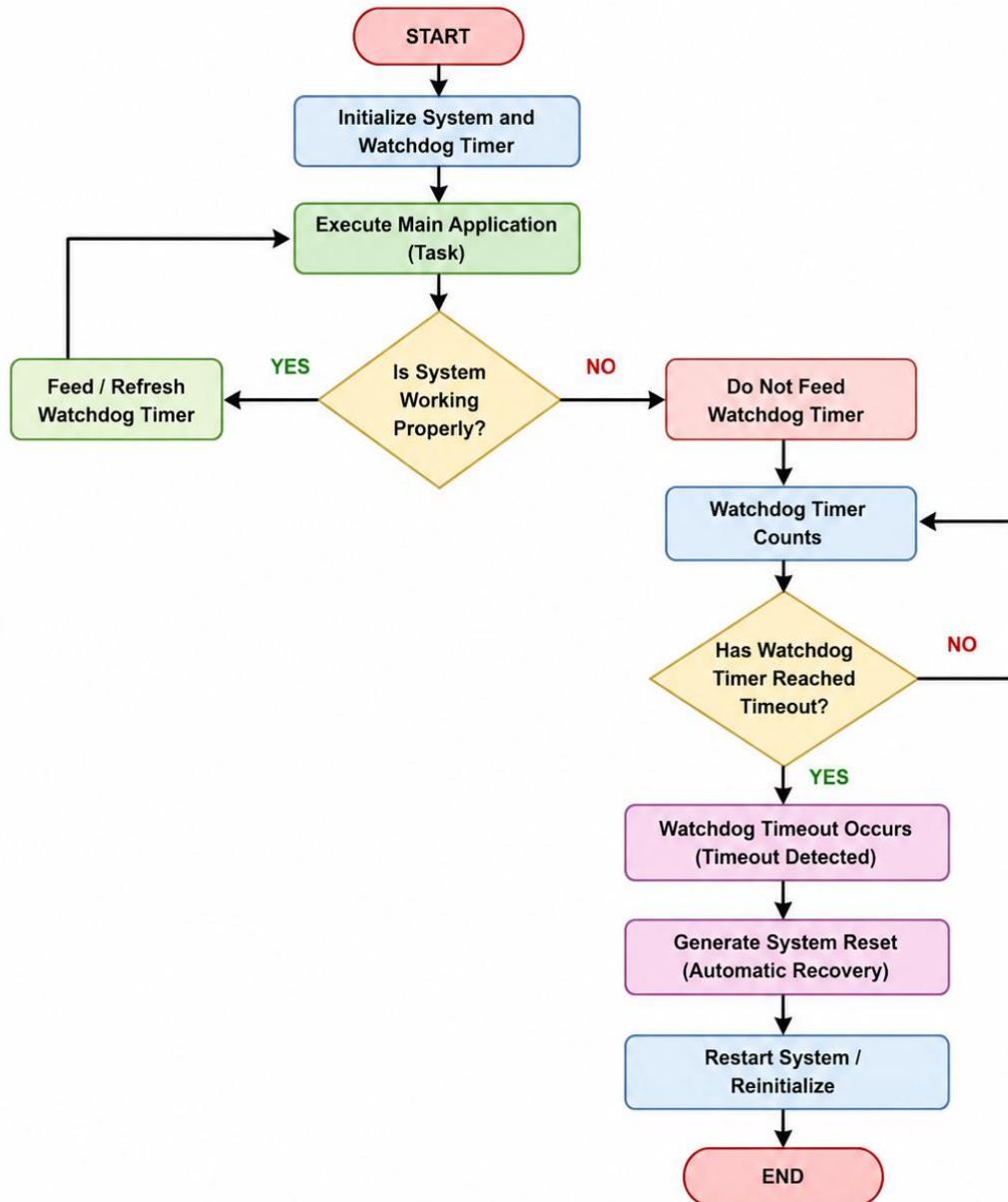
Example: ATM Cash Withdrawal System

Objective: To withdraw cash from an ATM machine.

Step No.	Algorithm Step	Real Life Example (ATM)		Explanation
1	Start		User approaches the ATM.	The process begins.
2	Insert ATM card		User inserts the ATM card into the machine.	ATM reads the card details.
3	Enter PIN		User enters the 4-digit PIN.	Verifies the user identity.
4	Verify PIN		ATM verifies the entered PIN.	- If correct → go to next step. - If incorrect → display error and terminate.
5	Select transaction type		User selects "Cash Withdrawal" option.	ATM shows available options.
6	Enter amount		User enters the amount to withdraw.	ATM checks account balance.
7	Check sufficient balance		ATM verifies if the account has sufficient balance.	- If sufficient → proceed. - If insufficient → display message and terminate.
8	Dispense cash		ATM dispenses the requested cash.	Cash is delivered to the user.
9	Display transaction success		ATM displays "Transaction Successful" message.	Confirms the successful transaction.
10	Eject card		ATM ejects the card.	User collects the card.
11	End		Transaction is completed.	The process ends.

Flowchart

Watchdog Timer Based Hardware Timeout Mechanism
Flowchart



Source Code

```
#include <stdio.h>

#define WATCHDOG_LIMIT 5

int watchdogCounter = 0;

/* Refresh watchdog timer */

void feedWatchdog(void)

{

    watchdogCounter = 0;

    printf("Watchdog Refreshed\n");

}

/* Check watchdog timeout */

void watchdogCheck(void)

{

    watchdogCounter++;

    if (watchdogCounter >= WATCHDOG_LIMIT)

    {

        printf("\n*****\n");

        printf("WATCHDOG TIMEOUT DETECTED\n");

        printf("System Reset Initiated\n");

        printf("Restarting Application...\n");

    }

}
```

```

printf("*****\n\n");

watchdogCounter = 0;

}

}

int main()

{

    int i;

    printf("-----\n");

    printf(" WATCHDOG TIMER SIMULATION\n");

    printf("-----\n");


    printf("\nSystem Boot Successful\n");


    printf("\n==== Normal Operation ==== \n");

    for(i = 1; i <= 10; i++)

    {

        printf("\nCycle %d\n", i);

        printf("Executing Task...\n");

        feedWatchdog();

        watchdogCheck();

```

```

    }

    printf("\n===== Software Failure =====\n");

    printf("Application entered infinite loop.\n");

    printf("Watchdog will not be refreshed.\n\n");

    for(i = 1; i <= 7; i++)

    {

        printf("Cycle %d\n", i);

        watchdogCheck();

    }

    printf("\nProgram Finished\n");

    return 0;

}

```

Check the Output

```

-----

WATCHDOG TIMER SIMULATION

-----

System Boot Successful

===== Normal Operation =====

Cycle 1

Executing Task...

```

Watchdog Refreshed

Cycle 2

Executing Task...

Watchdog Refreshed

...

===== Software Failure =====

Application entered infinite loop.

Watchdog will not be refreshed.

Cycle 1

Cycle 2

Cycle 3

Cycle 4

Cycle 5

WATCHDOG TIMEOUT DETECTED

System Reset Initiated

Restarting Application...

Cycle 6

Cycle 7

```
1 #include <stdio.h>
2
3 #define WATCHDOG_LIMIT 5
4
5 int watchdogCounter = 0;
6
7 /* Refresh watchdog timer */
8 void feedWatchdog(void)
9 {
10     watchdogCounter = 0;
11     printf("Watchdog Refreshed\n");
12 }
13
14 /* Check watchdog timeout */
15 void watchdogCheck(void)
16 {
17     watchdogCounter++;
18
19     if (watchdogCounter >= WATCHDOG_LIMIT)
20     {
21         printf("\n*****\n");
22         printf("WATCHDOG TIMEOUT DETECTED\n");
23         printf("System Reset Initiated\n");
24         printf("Restarting Application...\n");
25         printf("*****\n\n");
26
27         watchdogCounter = 0;
28     }
29 }
30
31 int main()
32 {
33     int i;
34     ...
35 }
```

WATCHDOG TIMER SIMULATION

System Boot Successful

==== Normal Operation ====

Cycle 1
Executing Task...
Watchdog Refreshed

Cycle 2
Executing Task...
Watchdog Refreshed

Cycle 3
Executing Task...
Watchdog Refreshed

Cycle 4
Executing Task...
Watchdog Refreshed

Cycle 5
Executing Task...
Watchdog Refreshed

Cycle 6
Executing Task...
Watchdog Refreshed

Cycle 7
Executing Task...
Watchdog Refreshed

```
31 int main()
32 {
33     int i;
34
35     printf("-----\n");
36     printf("WATCHDOG TIMER SIMULATION\n");
37     printf("-----\n");
38
39     printf("\nSystem Boot Successful\n");
40
41     printf("\n==== Normal Operation ==== \n");
42
43     for(i = 1; i <= 10; i++)
44     {
45         printf("\nCycle %d\n", i);
46         printf("Executing Task...\n");
47
48         feedWatchdog();
49         watchdogCheck();
50     }
51
52     printf("\n==== Software Failure ==== \n");
53     printf("Application entered infinite loop.\n");
54     printf("Watchdog will not be refreshed.\n\n");
55
56     for(i = 1; i <= 7; i++)
57     {
58         printf("Cycle %d\n", i);
59         watchdogCheck();
60     }
61
62     printf("\nProgram Finished\n");
63
64     return 0;
65 }
```

Executing Task...
Watchdog Refreshed

Cycle 9
Executing Task...
Watchdog Refreshed

Cycle 10
Executing Task...
Watchdog Refreshed

==== Software Failure ====
Application entered infinite loop.
Watchdog will not be refreshed.

Cycle 1
Cycle 2
Cycle 3
Cycle 4

WATCHDOG TIMEOUT DETECTED
System Reset Initiated
Restarting Application...

Cycle 5
Cycle 6
Cycle 7

Program Finished

...Program finished with exit code 0
Press ENTER to exit console.

Test Cases

Test Case	Description	Expected Result	Status
TC1	Normal execution	Watchdog refreshed successfully	Pass
TC2	Watchdog not refreshed	Timeout detected	Pass
TC3	System reset	Recovery message displayed	Pass

Test Case	Description	Expected Result	Status
TC4	Multiple execution cycles	Stable execution	Pass
TC5	Timeout boundary (5 cycles)	System reset occurs	Pass

Output Verification

The program output confirms that:

- The watchdog is refreshed during normal operation.
- No timeout occurs while the watchdog is refreshed.
- During software failure, the watchdog is not refreshed.
- A timeout is detected after the specified limit.
- The system reset message is displayed, demonstrating automatic recovery.

Time Complexity

$O(n)$

where n is the number of execution cycles.

Space Complexity

$O(1)$

The program uses only a few variables and a watchdog counter.

Embedded Concepts Used

- Embedded C Programming
- Watchdog Timer (WDT)
- Hardware Timeout Mechanism
- Fault Detection
- Automatic System Recovery
- Modular Programming

Hardware Abstraction

In actual microcontrollers, the watchdog timer is implemented as dedicated hardware. The application periodically refreshes the watchdog using hardware registers. If the processor hangs or enters an infinite loop, the watchdog expires and automatically resets the microcontroller. This project simulates the same behaviour using a software counter.

Real-Time Constraints

- The watchdog must be refreshed before the timeout period.
- Missing a refresh indicates that the application has become unresponsive.
- A timeout immediately triggers a system reset, ensuring reliable operation in real-time embedded systems.

Advantages

- Improves system reliability.
- Detects software failures.

- Recovers automatically from hangs.
- Prevents prolonged system crashes.
- Widely used in embedded applications.

Limitations

- Does not identify the root cause of software failures.
- Incorrect timeout settings may lead to unnecessary resets.
- Primarily intended for recovery rather than debugging.

Conclusion

The watchdog timer-based timeout mechanism was successfully implemented and tested using Embedded C. The program demonstrates that regular watchdog refreshes allow normal operation, while failure to refresh the watchdog triggers a timeout and simulated system reset. This mechanism improves the reliability and fault tolerance of embedded systems by automatically recovering from software failures.

Title

Construct an Embedded C program using structures and function pointers to represent modular sensor interfaces.

Problem Understanding

In embedded systems, multiple sensors (Temperature, Humidity, Light, etc.) are connected to a microcontroller.

Instead of writing separate code for every sensor, structures are used to store sensor information and function pointers are used to call the correct sensor function dynamically.

This makes the program modular, reusable, and easy to extend.

2. Objective

To design an Embedded C program that demonstrates modular sensor interfaces using structures and function pointers for different sensors.

3. Real-Life Example

Smart Greenhouse Monitoring System

A greenhouse contains multiple sensors:

- 🌡️ Temperature Sensor
- 💧 Humidity Sensor
- ☀️ Light Sensor

Each sensor has:

- Sensor Name
- Sensor ID
- Function to read data

Instead of writing different code for every sensor, the controller stores all sensors inside a structure and calls each sensor using a function pointer.

This is exactly how many real embedded systems are designed.

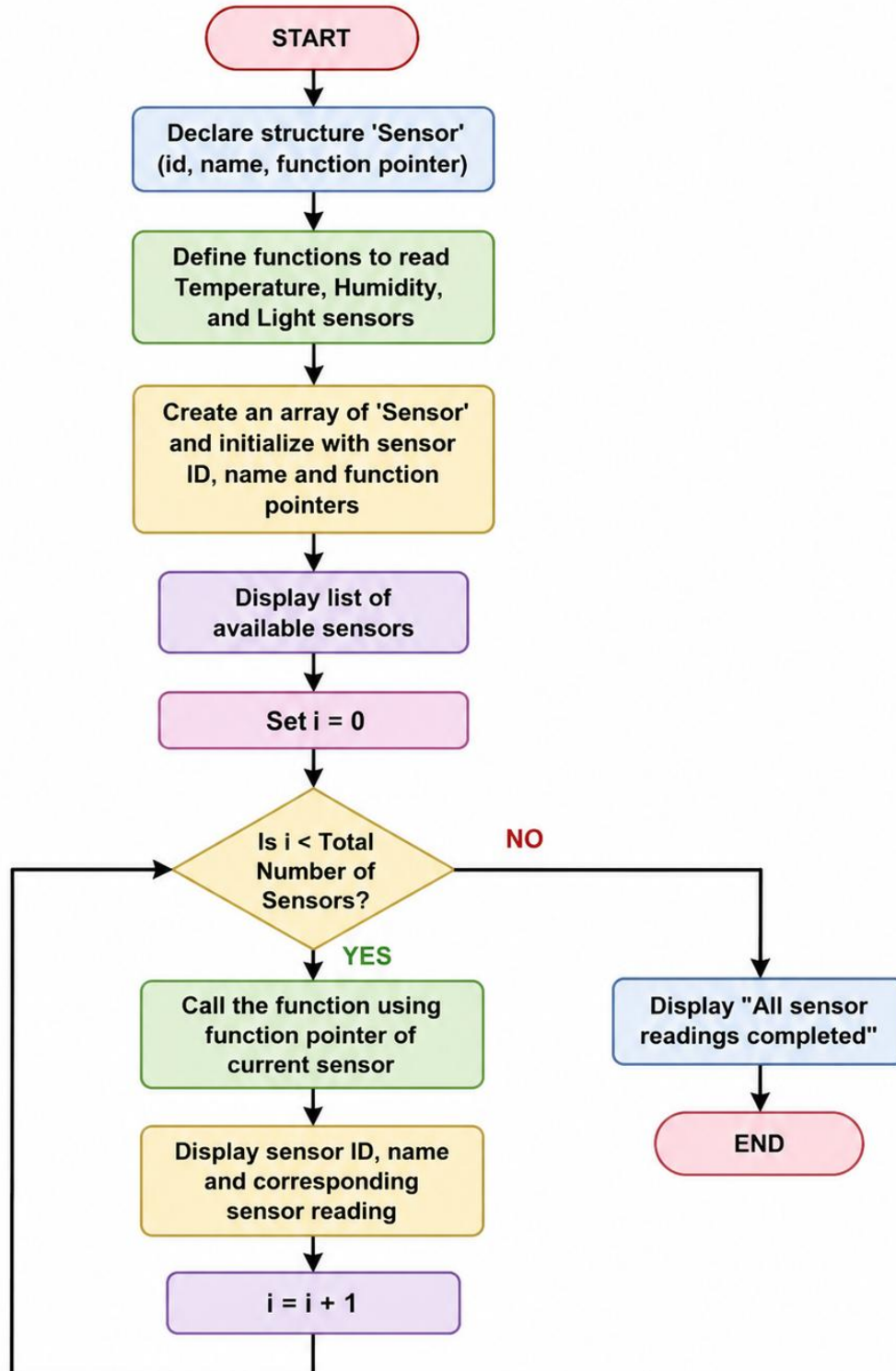
Algorithm

1. Start the program.
2. Create a structure to store sensor information.
3. Define functions to read Temperature, Humidity, and Light sensors.
4. Assign the corresponding function pointer to each sensor.
5. Store sensor name and ID.
6. Display available sensors.
7. Call each sensor function through its function pointer.
8. Display sensor readings.
9. Stop the program.

Flowchart

FLOWCHART

Modular Sensor Interfaces using Structures and Function Pointers



Embedded C Program (main.c)

```
#include <stdio.h>

/* Function Prototypes */

void readTemperature();

void readHumidity();

void readLight();

/* Structure Definition */

struct Sensor

{

    int id;

    char name[20];

    void (*readSensor)();

};

/* Temperature Sensor */

void readTemperature()

{

    printf("Temperature : 30 C\n");

}

/* Humidity Sensor */

void readHumidity()

{

    printf("Humidity : 65 %%\n");
```

```

}

/* Light Sensor */

void readLight()

{

    printf("Light Intensity : 850 Lux\n");

}

int main()

{

    struct Sensor sensors[3] =

    {

        {1,"Temperature",readTemperature},

        {2,"Humidity",readHumidity},

        {3,"Light",readLight}

    };

    int i;

    printf("==== Modular Sensor Interface ====\n\n");

    for(i=0;i<3;i++)

    {

        printf("Sensor ID : %d\n",sensors[i].id);

        printf("Sensor Name : %s\n",sensors[i].name);

        sensors[i].readSensor();

        printf("\n");
    }

```

```

    }

    return 0;

}

```

```

main.c
1  #include <stdio.h>
2
3  /* Function Prototypes */
4  void readTemperature(void);
5  void readHumidity(void);
6  void readLight(void);
7
8  /* Structure Definition */
9  struct Sensor
10 {
11     int id;
12     char name[20];
13     void (*readSensor)(void);
14 };
15
16 /* Function to read Temperature Sensor */
17 void readTemperature(void)
18 {
19     printf("Temperature : 30 C\n");
20 }
21
22 /* Function to read Humidity Sensor */
23 void readHumidity(void)
24 {
25     printf("Humidity : 65 %\n");
26 }
27
28 /* Function to read Light Sensor */
29 void readLight(void)
30 {
31     printf("Light Intensity : 850 Lux\n");
32 }
33
34 int main(void)
35 {

```

```

=====
MODULAR SENSOR INTERFACE USING STRUCTURES
AND FUNCTION POINTERS
=====

Sensor ID    : 1
Sensor Name  : Temperature
Temperature  : 30 C
-----

Sensor ID    : 2
Sensor Name  : Humidity
Humidity     : 65 %
-----

Sensor ID    : 3
Sensor Name  : Light
Light Intensity : 850 Lux
-----

All Sensor Readings Completed Successfully.

...Program finished with exit code 0
Press ENTER to exit console.

```

Test Cases

Test Case	Input	Expected Output	Status
TC1	Temperature Sensor	Temperature displayed	Pass
TC2	Humidity Sensor	Humidity displayed	Pass
TC3	Light Sensor	Light value displayed	Pass
TC4	All Sensors	All values displayed	Pass
TC5	Modular function call	Correct function executed	Pass

Output Verification

Feature	Result
Structure Created	Correct
Function Pointer Assigned	Correct
Sensor Read Successfully	Correct
Output Displayed	Correct

Time Complexity

$O(n)$ where n = number of sensors

Space Complexity

$O(n)$ Stores n sensor structures.

Embedded Concepts Used

- Embedded C
- Structures
- Function Pointers
- Modular Programming
- Sensor Interfaces
- Hardware Abstraction

Hardware Abstraction

Each sensor is represented as a structure with its own reading function. The main program does not directly interact with hardware-specific code; instead, it calls the appropriate function through a function pointer. This modular approach simplifies adding or replacing sensors without changing the overall application logic.

Real-Time Constraints

- Sensors must be read periodically.
- Each sensor function should execute quickly to maintain real-time performance.
- Function pointers allow flexible sensor management without increasing program complexity.

Advantages

- Modular design.

- Easy to add new sensors.
- Code reusability.
- Easy maintenance.
- Better scalability.

Limitations

- Uses slightly more memory than a single-function design.
- Requires understanding of function pointers.
- Function pointer misuse may lead to runtime errors.

Conclusion

The program successfully demonstrates a modular sensor interface using structures and function pointers in Embedded C. Each sensor is represented independently and accessed through its associated function pointer, making the design flexible, reusable, and suitable for embedded applications that involve multiple sensors.